AI-Powered Secure Container Delivery and Vulnerability Detection Platform

Problem Statement Category: Cloud - Secure container delivery

1. Problem Statement

Containers are now the standard way to package and ship software, but images are often built from public base layers, carry outdated packages, and are deployed with insecure defaults. Teams struggle to find, rank and fix these issues before release, which leads to unsafe deployments and production exposure.

2. Abstract

The AI-Powered Secure Container Delivery and Vulnerability Detection Platform aims to improve the security and reliability of software container deployment. The platform analyzes container images, identifies potential vulnerabilities, detects insecure configurations, and assigns risk levels to help developers prioritize security issues. By integrating AI-assisted vulnerability analysis with automated security checks, it provides actionable recommendations before deployment. The system also supports secure image management, vulnerability reporting, and continuous monitoring of container security. This solution helps development teams reduce security risks, prevent unsafe deployments, and build a more reliable software delivery pipeline.

3. Key Features

- Container image analysis: scans OS packages, libraries and layers for known vulnerabilities.
- Insecure configuration detection: root users, exposed ports, hard-coded secrets, excessive privileges.
- AI-assisted risk scoring: assigns risk levels so developers can prioritize the most critical issues.
- Actionable recommendations: suggests fixed versions, hardened base images and configuration changes.
- Secure image management: signed, versioned images stored in a trusted registry.
- Vulnerability reporting and continuous monitoring of deployed containers.

4. System Architecture

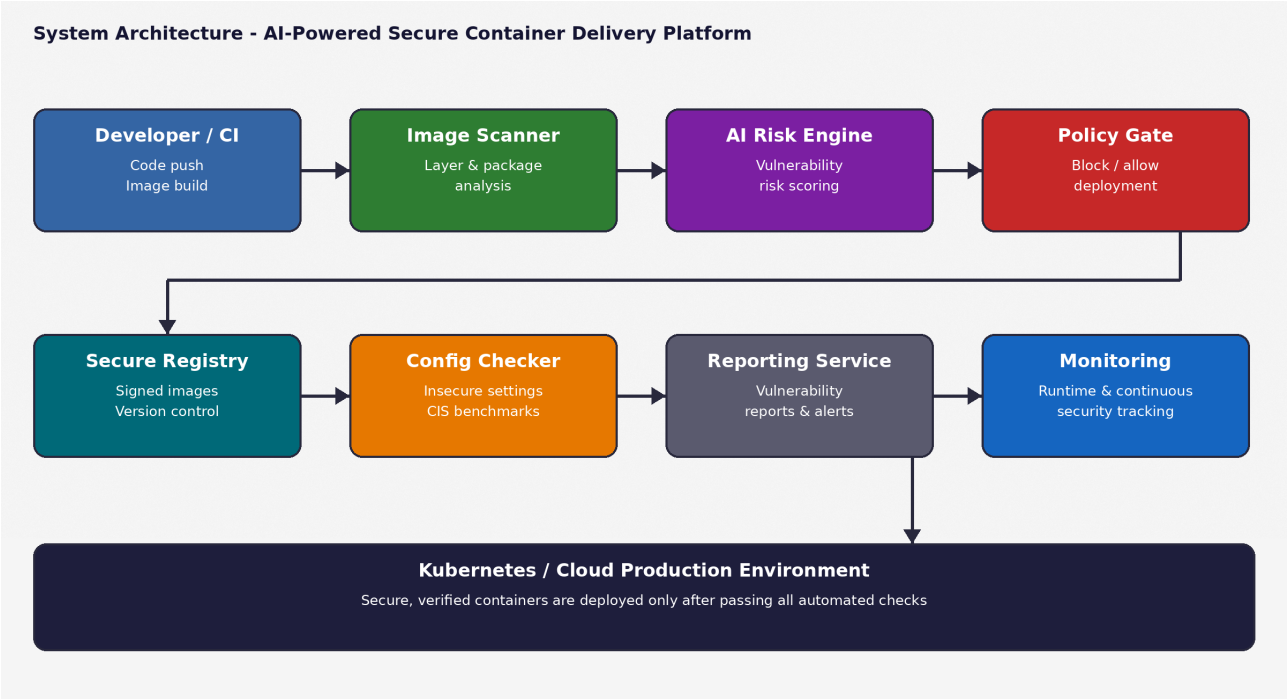


Figure 1: High-level architecture showing the build, scan, risk-analysis, policy-gate and deployment stages.

5. Workflow

1. Developer pushes code; the CI pipeline builds a container image.
2. The scanner inspects image layers, packages and configuration files.
3. The AI risk engine correlates findings with vulnerability databases and assigns risk levels.
4. The policy gate blocks images that violate security policy and returns fix recommendations.
5. Approved images are signed and stored in the secure registry.
6. Images are deployed to the cloud / Kubernetes environment.
7. Continuous monitoring re-scans running images and raises alerts for newly disclosed vulnerabilities.

6. Tech Stack Planned

Layer	Technology
Frontend dashboard	React, Tailwind CSS
Backend / API	Python (FastAPI)
Image scanning	Trivy, Grype, Dockerfile linting (Hadolint)
AI / Risk analysis	Python ML models, LLM-assisted recommendations
CI/CD integration	GitHub Actions / Jenkins
Registry & signing	Harbor / Docker Registry, Cosign
Deployment	Docker, Kubernetes, Cloud (AWS / Azure / GCP)

Database	PostgreSQL
Monitoring	Prometheus, Grafana, Falco

7. Expected Outcome

Development teams reduce security risk, prevent unsafe deployments, and gain a more reliable and automated software delivery pipeline.